

CSE4354 Real-time Operating Systems

Fall 2026 Mini Project (Memory Protection Unit and Memory Manager)

1 Overview

The goal of this assignment is to introduce discrete privileged and unprivileged operation modes, add exception handling and diagnostics, provide memory management, and implement memory protection that will be used in the RTOS project.

2 Hardware Description

Microcontroller:

An ARM M4F core (TM4C123GH6PMI microcontroller) is required.

Serial interface:

If using the EK-TM4C123GXL evaluation board, then the UART0 tx/rx pair is routed to the ICDI that provides a virtual COM port through a USB endpoint.

3.3V supply:

The circuit is powered completely from the 3.3V regulator output on the evaluation board.

3 Software Requirements

so even before rtos(), we gotta write assembly to write pgs8 set asp(1)

A virtual COM port using a 115200 baud, 8N1 protocol with no hardware handshaking is used to communicate with this project.

For this design, you must set the ASP bit in the CONTROL register so that the thread code uses the PSP (the handler mode will always use the MSP). Make sure to place an instruction barrier instruction (ISB) after the write to the CONTROL register so that instructions use the correct stack.

The software must provide the following functions. As in the shell interface, you cannot call C library functions.

LR register tell us if register 8 is pushed or 25. so we have to conserve for running process

interrupt isr happens right at next clock. it doesnt have clock time to push registers

If a hard fault exception is invoked, display "Hard fault in thread PID", where PID will be a variable provided by the OS. Or now, just use a variable named pid. Also, provide the value of the PSP, MSP, and all fault flags (in hex). Also, print the offending instruction. Display the process stack dump (xPSR, PC, LR, R0-3, R12).

so is our problem 1. isr cant push registers? or 2. isr pushed registers and overwrite LR running process to return to caller.

If an MPU exception is invoked, display "MPU fault in thread PID", where PID will be a variable provided by the OS. Or now, just use a variable named pid. Also, provide the value of the PSP, MSP, and mfault flags (in hex). Also, print the offending instruction and data addresses. Display the process stack dump (xPSR, PC, LR, R0-3, R12). Clear the MPU fault pending bit and trigger a pendsv ISR call.

If a bus fault exception is invoked, display "Bus fault in thread PID", where PID will be a variable provided by the OS. Or now, just use a variable named pid.

uint32_t* pop = getPsp();
uint32_t R0 = *pop <- simple as this

If a usage fault exception is invoked, display "Usage fault in thread PID", where PID will be a variable provided by the OS. Or now, just use a variable named pid.

make sure set psp before we do all this

If a pendsv exception is invoked, display "Pendsv in thread PID". If the MPU DERR or IERR bits are set, clear them and display the message "memory protection called this handler".

Understand how all of these exceptions are triggered by code. Using the circuit for the main project detailed in class, assign each of the push buttons to invoke intentional errors to trigger the exceptions above. Make sure the push buttons are clearly labeled with the number 0-6 in order and also the exception being generated for as many buttons as are needed.

after we set asp(1), we can make below assembly code so simple, cuz psp is point right at r0

You will also need to create a simple memory manager than allocates memory from the global heap. The function prototype is

```
getsp(); in assembly  
{  
  mov RN, SP  
  bc LR  
}
```

```
uint32_t* getR0()  
{  
  BX LR  
}
```

```
getpsp  
mxs R0, PSP  
BX LR
```

32k ram, 13ish stacks, no v-table so no way to handle fragmentation

print usage fault too,

embded msp only, privileged always. how did program worked?
how does memory look like?

void * malloc_from_heap (int size_in_bytes) 512B then stack point 2000 0000 to 2000 0200 $16^2 * 2 = 512b$
o/s is 4KB? then 2000 0000 to 2000 0700

The name is chosen to not conflict with the malloc function in C which is not implemented for our OS.

For Honors contract students, you will implement the memory manager as follows:

The memory allocated will be rounded up to the nearest multiple of 512B if the allocation requested is $\leq 512B$ and rounded up to the nearest multiple of 1024B otherwise. The memory address of all allocated memory chunks will be aligned to a multiple of 512 for 512B allocations and a multiple of 1024 for $N \cdot 1024B$ allocations. The mix of 1024B and 512B allocations is a design decision. At least 5 regions must be used.

For most students, you will implement the memory manager as follows:

The memory allocated will be rounded up to the nearest multiple of 1024B if the allocation requested is $\leq 1024B$ and rounded up to the nearest multiple of 1024B otherwise. The memory address of all allocated memory chunks will be aligned to a multiple of 1024.

one way to prompt bus fault. create lots of static variable
then it will underflow down

Implement free_to_heap(void *p) function as well.

The intent is to re-compile between these steps as features are added and verification is performed. It is suggested that you not skip steps as an error would have multiple possible sources. Make sure exception handlers are complete before proceeding.

Study in detail in the ARM documentation the relative prioritization of the MPU regions before starting the following steps:

global pointer to function directories.

To configure the MPU regions, follow these steps:

have heap start at 2000 1000, and how to tell c
that? tm4c.ccs file isr, stack top, initial value written at
of chip. loaded in pc at first instrction

Overall Access (optional background rule)

To make the following steps easier to debug, add an all memory background rule that allows RW access to all memory for both privileged and unprivileged code. You can remove access for the unprivileged code later or leave it in the code (see the note in the peripheral access section on this).

Flash Access

when you create thread (*ptr to function, priority, size of stack)
arguments list we need. we will know exact size of stack. its not variable. call func()?

Implement a function, void allowFlashAccess(void), that creates a full-access MPU aperture for flash with RWX access for both privileged and unprivileged access.

Call allowFlashAccess(). While in privileged mode, verify the program still runs as expected when turning on MPU with flash access control.

stack of threads will be allocated in heap

Temporarily switch to unprivileged mode and verify that you can access flash and run code.

Revert to privileged mode before continuing.

beginning of stack + size of stack then start of instruction
thread must have stacks at least for interrupts

Peripheral Access

Implement a function, void allowPeripheralAccess(void), that creates a full-access MPU aperture to peripherals and peripheral bitbanded addresses with RW access for privileged and unprivileged access. If you choose to leave the all access rule, then this is not needed.

Call allowFlashAccess() and allowPeripheralAccess(). While in privileged mode, verify that the program still runs as expected when turning on MPU with flash, RAM and peripheral access control.

Temporarily switch to unprivileged mode and verify that you can access flash (run code) and r/w memory.

Revert to privileged mode before continuing.

os need to know who this memory is allocated to and size
when claim back, erase data for security, PID

hijack upper bits of pointer

to write how many chunks we using, physical addr

bookkeeping which will be stored at static variables o/s kernel not allowed to Read by any thread, if did -> fault
heap alloc table for every 1KiB
search for contiguous. array return null if no contiguous => fragmentation

SRAM Access (All)

Create a function, void setupSramAccess(void), that creates multiple MPU regions to cover the 32KiB SRAM (each MPU region covers 8KiB or 4 KiB, with 8 subregions of 1KiB or 512B each with RW access for privileged mode and no access for unprivileged mode. It may be useful to disable the subregions to start.

Create a function uint64_t srdBitMask = createNoSramAccessMask(void) returns the values of the srdBits to allow no access to SRAM.

Create a function applySramAccessMask(uint64_t srdBitMask) that applies the SRD bits to the MPU regions. This function will only be called in privileged mode.

Create a function void addSramAccessWindow(uint64_t *srdBitMask, uint32_t *baseAdd, uint32_t size_in_bytes) that adds access to the requested SRAM address range.
Call allowFlashAccess(), allowPeripheralAccess(), setupSramAccess(), createNoSramAccess(), and setSramAccessWindow((uint32_t *)0x20000000, 32768).

While in privileged mode, verify that the program still runs as expected when turning on MPU with flash, RAM and peripheral access control.

Temporarily switch to unprivileged mode and verify that you can access flash (run code) and r/w memory.

Revert to privileged mode before continuing.

SRAM Access (Restricted)

Call uint32_t *p = malloc_from_heap(1024)

Call allowFlashAccess(), allowPeripheralAccess(), setupSramAccess(), createNoSramAccess(), and setSramAccessWindow(p, 1024) to create access to the newly allocated memory.

Dereference the pointer and write to the address (*p = value).

While in privileged mode, verify you can still access ram in the allocated range of SRAM.

Switch to unprivileged mode and verify that you can access flash (run code) and r/w memory from the peripherals.

Call free_to_heap(p)

In unprivileged mode, dereference the pointer and write to the address (*p = value) and verify there is now a fault.

While in privileged mode, verify you can still access ram in the allocated range of SRAM.

Switch to unprivileged mode and verify that you can access flash (run code) and r/w memory from the peripherals.

While in unprivileged mode, verify that accessing SRAM outside of the allocated range is not allowed (you will see an MPU exception with information printed about the errant read/write from memory).

4 Testing

Your code will be tested in the ERB 125 lab by the grader.

5 Deadlines

The assignment is due on the date and at the time indicated in the syllabus. This is an individual project, so do not exchange or share code with others.

6 Safety Issues

When utilizing the lab, please observe all safety rules as stated in the syllabus.

Have fun!